

Documento de Arquitetura – Projeto de Testes de Performance com K6

1. Visão Geral

Este documento descreve a **arquitetura do projeto de testes automatizados de performance** utilizando **K6**, agora **adaptada para a API real disponível no repositório**:

👉 <https://github.com/juliodelimas/pgats-2025-02-base>

O objetivo do projeto é avaliar o comportamento de uma **API REST local em Node.js** sob diferentes níveis de carga, simulando fluxos reais de negócio (cadastro, autenticação e checkout), demonstrando boas práticas de engenharia de testes de performance e atendendo integralmente aos requisitos acadêmicos do desafio.

2. API Alvo

2.1 API escolhida

pgats-2025-02-base – API REST desenvolvida em Node.js, utilizada como base prática para testes automatizados.

2.2 Justificativa da escolha

A API foi escolhida por apresentar características ideais para testes de performance acadêmicos:

- Possui **fluxo completo de negócio** (cadastro → login → checkout)
- Implementa **autenticação via JWT**
- Contém **endpoints protegidos** que exigem token
- Permite operações **GET e POST**
- Pode ser executada localmente, garantindo controle total do ambiente
- Evita limitações de rate limit comuns em APIs públicas

Essa escolha permite testar não apenas endpoints isolados, mas **fluxos reais de aplicação**, o que enriquece a análise de performance.

3. Endpoints Utilizados nos Testes

Considerando a API do repositório, os seguintes endpoints são utilizados:

Cadastro de Usuário

POST /api/users/register

Responsável por criar usuários dinamicamente para os testes.

Login de Usuário

POST /api/users/login

Responsável por autenticar o usuário e retornar o **token JWT**, que será reaproveitado nos fluxos seguintes.

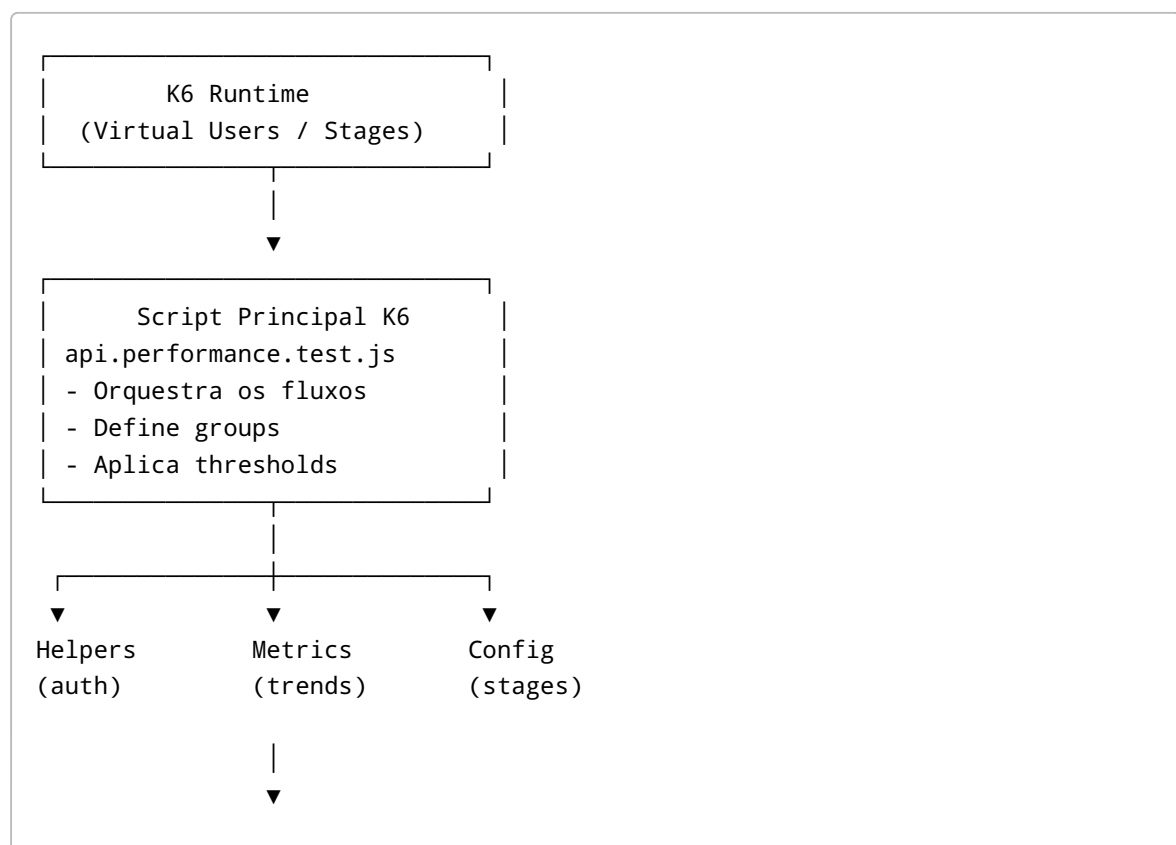
Checkout

POST /api/checkout

Endpoint protegido que exige token JWT no header `Authorization`.

4. Visão Arquitetural do Projeto de Testes

A arquitetura segue o princípio de **separação de responsabilidades**, facilitando leitura, manutenção e avaliação acadêmica.



5. Estrutura de Pastas e Responsabilidades

```
test/
├── k6/
│   ├── helpers/      # Funções reutilizáveis (login, headers, etc.)
│   ├── data/         # Massa de dados para data-driven testing
│   ├── metrics/       # Métricas customizadas (Trends)
│   ├── config/        # Configuração de carga e thresholds
│   └── tests/         # Scripts de teste e cenários
```

5.1 helpers/

Contém **lógicas reutilizáveis**, evitando duplicação de código.

Exemplo: - `auth.helper.js`: realiza login no endpoint `/api/users/login` e retorna o token JWT.

5.2 data/

Armazena arquivos utilizados para **Data-Driven Testing**.

Exemplo: - `users.json`: usuários base utilizados para login ou registro.

5.3 metrics/

Define **métricas customizadas** usando `Trend`, permitindo análises específicas além das métricas padrão do K6.

Exemplos: - Tempo de resposta do fluxo de checkout - Tamanho do payload das respostas

5.4 config/

Centraliza configurações globais relacionadas à carga e critérios de sucesso.

Exemplo: - `stages.js`: definição de ramp-up, carga sustentada e ramp-down - Thresholds reutilizáveis

5.5 tests/

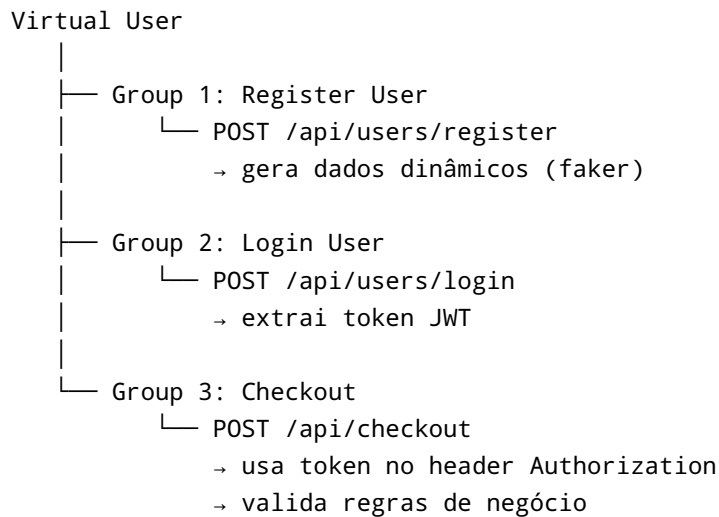
Contém o **script principal de testes**, responsável por orquestrar os fluxos de performance.

Exemplo: - `api.performance.test.js`

6. Fluxo de Execução do Teste

6.1 Fluxo Funcional Simulado

O fluxo executado por cada Virtual User simula um comportamento real de usuário:



Cada etapa é isolada em um **Group**, facilitando leitura do código e análise dos resultados no relatório.

7. Estratégia de Carga (Stages)

A carga é configurada para simular um cenário realista de uso da API:

Fase	Descrição	Objetivo
Ramp-up	Aumento gradual de VUs	Evitar picos artificiais
Load	Carga sustentada	Avaliar estabilidade e SLA
Ramp-down	Redução gradual	Encerramento controlado

8. Estratégia de Dados

8.1 Data-Driven Testing

- Uso de arquivos JSON para dados base
- Seleção de usuários baseada no VU (`__VU`)
- Permite variação de cenários

8.2 Geração Dinâmica de Dados (Faker)

- Geração de usuários únicos
 - Criação dinâmica de payloads de checkout
 - Evita colisões e simula dados reais
-

9. Autenticação e Segurança

- Autenticação via **JWT**
 - Token obtido no login
 - Token reaproveitado em requisições protegidas
 - Header `Authorization: Bearer <token>`
-

10. Observabilidade e Critérios de Sucesso

Métricas Monitoradas

- `http_req_duration`
- `custom_response_time`
- `custom_payload_size`

Critérios (Thresholds)

- 95% das requisições abaixo do tempo máximo definido
 - Taxa de erro funcional inferior a 5%
-

11. Execução do Projeto

Execução Local

```
k6 run -e BASE_URL=http://localhost:3000 test/k6/tests/
api.performance.test.js
```

Relatórios

- Geração de relatório HTML ou JSON
 - Compatível com execução local e CI/CD
-

12. Benefícios da Arquitetura

- Arquitetura clara e didática para avaliação acadêmica
- Total aderência aos conceitos exigidos pelo desafio
- Código modular, reutilizável e escalável
- Fácil adaptação para outras APIs REST

13. Próximos Passos

1. Implementar scripts K6 adaptados aos endpoints reais 📌
 2. Atualizar README com exemplos reais da API
 3. Executar testes de performance
 4. Gerar relatórios e publicar no GitHub
-

Implementação dos Scripts K6 (Código)

A seguir está a implementação completa do projeto de testes de performance conforme a arquitetura definida, utilizando a API **pgats-2025-02-base**.

1. Configuração de Carga



test/k6/config/stages.js

```
export const stages = [
  { duration: '30s', target: 5 }, // ramp-up
  { duration: '1m', target: 10 }, // carga sustentada
  { duration: '30s', target: 0 }, // ramp-down
];

export const defaultThresholds = {
  http_req_duration: ['p(95)<1500'],
  checks: ['rate>0.95'],
};
```

2. Métricas Customizadas



test/k6/metrics/trends.js

```
import { Trend } from 'k6/metrics';

export const responseTimeTrend = new Trend('custom_response_time');
export const payloadSizeTrend = new Trend('custom_payload_size');
```

3. Helper de Autenticação

 test/k6/helpers/auth.helper.js

```
import http from 'k6/http';
import { check } from 'k6';

const BASE_URL = __ENV.BASE_URL || 'http://localhost:3000';

export function registerUser(user) {
  const res = http.post(`${BASE_URL}/api/users/register`,
    JSON.stringify(user), {
      headers: { 'Content-Type': 'application/json' },
    });

  check(res, {
    'user registered (201)': (r) => r.status === 201 || r.status === 200,
  });
}

export function loginUser(email, password) {
  const res = http.post(`${BASE_URL}/api/users/login`, JSON.stringify({
    email, password }), {
    headers: { 'Content-Type': 'application/json' },
  });

  check(res, {
    'login success (200)': (r) => r.status === 200,
    'token returned': (r) => r.json('token') !== undefined,
  });

  return res.json('token');
}
```

4. Massa de Dados

 test/k6/data/users.json

```
[
  { "name": "User A", "email": "usera@test.com", "password": "123456" },
  { "name": "User B", "email": "userb@test.com", "password": "123456" }
]
```

5. Script Principal de Teste



test/k6/tests/api.performance.test.js

```
import http from 'k6/http';
import { check, group, sleep } from 'k6';
import { stages, defaultThresholds } from '../config/stages.js';
import { responseTimeTrend, payloadSizeTrend } from '../metrics/trends.js';
import { registerUser, loginUser } from '../helpers/auth.helper.js';

const users = JSON.parse(open('test/k6/data/users.json'));
const BASE_URL = __ENV.BASE_URL || 'http://localhost:3000';

export const options = {
  stages,
  thresholds: {
    ...defaultThresholds,
    custom_response_time: ['p(95)<2000'],
  },
};

function fakerUser() {
  const id = Math.floor(Math.random() * 100000);
  return {
    name: `User ${id}`,
    email: `user${id}@test.com`,
    password: '123456',
  };
}

export default function () {
  const baseUser = users[__VU % users.length];
  const user = Math.random() > 0.5 ? fakerUser() : baseUser;

  group('Register User', () => {
    registerUser(user);
  });

  let token;

  group('Login User', () => {
    token = loginUser(user.email, user.password);
  });

  const headers = {
    headers: {
      Authorization: `Bearer ${token}`,
      'Content-Type': 'application/json',
    },
  };
};
```



```
group('Checkout', () => {
  const payload = JSON.stringify({
    items: [{ productId: 1, quantity: 1 }],
    freight: 20,
    paymentMethod: 'boleto',
  });

  const res = http.post(`${BASE_URL}/api/checkout`, payload, headers);

  check(res, {
    'checkout success (200)': (r) => r.status === 200,
  });

  responseTimeTrend.add(res.timings.duration);
  payloadSizeTrend.add(res.body.length);
});

sleep(1);
}
```

6. Execução dos Testes

```
k6 run -e BASE_URL=http://localhost:3000 test/k6/tests/
api.performance.test.js
```

Gerar relatório HTML

```
k6 run --out html=report.html -e BASE_URL=http://localhost:3000 test/k6/
tests/api.performance.test.js
```

7. Resultado Esperado

- Execução do fluxo completo: **register** → **login** → **checkout**
- Uso de token JWT reaproveitado
- Métricas e thresholds validados
- Relatório pronto para avaliação acadêmica

Implementação concluída conforme a arquitetura definida.